Foundations of Cybersecurity

Project Report

Bacciottini Leonardo
Paolini Emilio

ACADEMIC YEAR 2019/2020

Contents

1 Introduction **2**

2 Instructions **3**
2.1 Starting the server 3
2.2 Opening a client 3

3 Protocols **6**
3.1 FRAH Protocol 6
3.2 Client-Server communication 7
3.2.1 Symmetric encryption 8
3.2.2 Four in a Row Communication Protocol (FRCP) . . . 9
3.3 P2P Handshake Protocol 11

4 In-game communication **13**

5 Ban Logic **14**
5.1 FRAH Protocol 14
5.2 P2P Handshake Protocol 17

1 Introduction

The goal of the project is to implement a secure client-server four-in-a-row. Clients are all pre-registered at the server, so no registration mechanism has been implemented.
When the client starts it asks the user if he/she wants to be a challenger or wants to wait for a challenge. If the user chooses the first option, then he/she can send a challenge to another user who chose the second option. Then, the client that receives a challenge can either accept or refuse it. If the user accepts the challenge, the game starts. The application also handles possible error situation, for example if a user challenges another challenger.

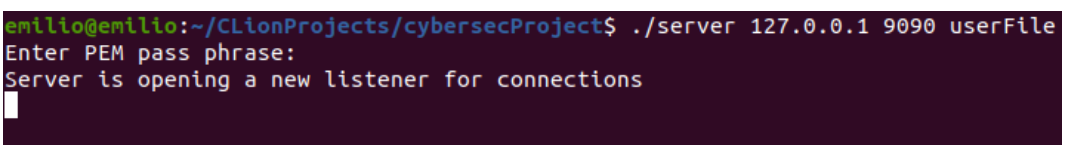
2 Instructions

2.1 Starting the server

To make the application working, the first thing to do is to open the server. To start the server, the following steps must be performed:

1. Go to the folder where there is the makefile.
2. Run "make" command. 2 files will be created: "client" and "server".
3. Finally, to start the server all you have to do is to run the following command: ./server $ip_address $port $userFile

Once started, the password to read the private key of the server must be inserted. Then the server will start listening for new connections from the clients, as showed in the following picture:

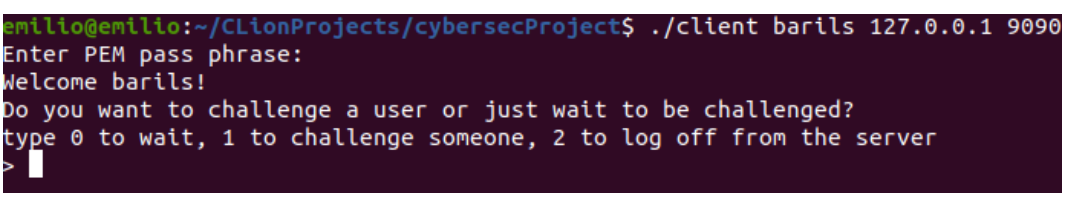


2.2 Opening a client

Once the server is running, clients can connect to it. To open a new client, the following steps must be perfomed:

1. Step 1-2 of server instructions.
2. Then, to start the client the following command must be typed: ./client $username $serverIP $serverPort

If client can authenticate, then the following screen will appear:



At this point a client can choose to be a challenger or to wait for a challenge by another user. In the first scenario, a list of active users will be sent from the server to the client:

```
emilio@emilio:~/CLionProjects/cybersecProject$ ./client barils 127.0.0.1 9090
Enter PEM pass phrase:
Welcome barils!
Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
> 1

Here's a list of active users. Choose one to challenge!
1) barils (you)
2) gabrysse
Type the user you want to challenge and press Enter >
```

Then, the client can choose the player that he/she wants to challenge and a request will be sent to the other.

```
emilio@emilio:~/CLionProjects/cybersecProject$ ./client barils 127.0.0.1 9090
Enter PEM pass phrase:
Welcome barils!
Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
> 1

Here's a list of active users. Choose one to challenge!
1) barils (you)
2) gabrysse
Type the user you want to challenge and press Enter > gabrysse
Challenging gabrysse
Waiting for a response by the challenged user...
```

Instead, if the client chooses to wait for a challenge, the following screen will appear:

```
emilio@emilio:~/CLionProjects/cybersecProject$ ./client gabrysse 127.0.0.1 9090
Enter PEM pass phrase:
Welcome gabrysse!
Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
> 0
Waiting for someone to challenge you...
```

If there isn't any request for a challenge in 20 seconds, then the user will be asked again to choose about his/her role:

```
emilio@emilio:~/CLionProjects/cybersecProject$ ./client gabrysse 127.0.0.1 9090
Enter PEM pass phrase:
Welcome gabrysse!
Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
> 0
Waiting for someone to challenge you...

Ops! Seems like no one challenged you. Try again if you want!

Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
>
```

Instead, if a challenge arrives within 20 seconds, a message asking the user if he/she wants to accept request will appear:

```
Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
> 0
Waiting for someone to challenge you...
You receive a challenge from barils
Type 1 to accept, 0 otherwise >
```

Once accepted, the playfield will appear on both the clients:

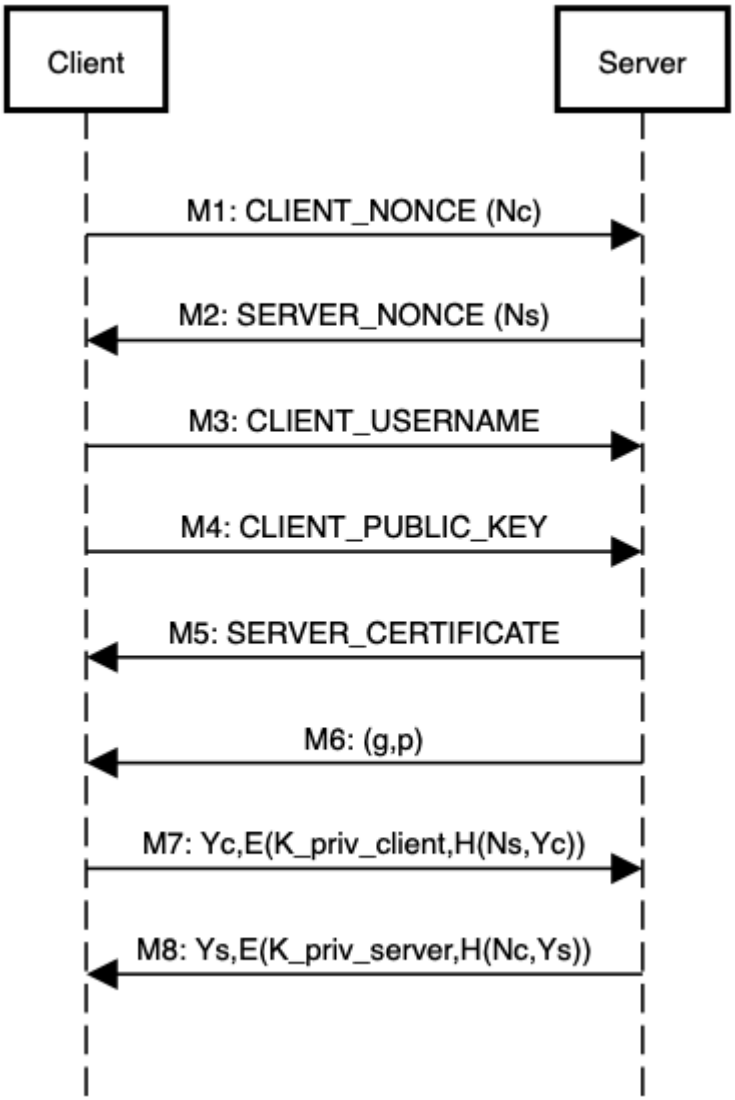
```
emilio@emilio:~/CLionProjects/cybersecProject$ ./client gabrysse 127.0.0.1 9090
Enter PEM pass phrase:
Welcome gabrysse!
Do you want to challenge a user or just wait to be challenged?
type 0 to wait, 1 to challenge someone, 2 to log off from the server
> 0
Waiting for someone to challenge you...
You receive a challenge from barils
Type 1 to accept, 0 otherwise > 1
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |
  0   1   2   3   4   5   6
In which column do you want to insert the token?
>
```

At the end of the game, both the clients can choose their roles again.

3 Protocols

3.1 FRAH Protocol

The FRAH protocol(Four-in-a-Row Authentication Handshake) is the protocol used by the client and the server to authenticate. Indeed, when the client starts, it has to authenticate with the server using public keys. Our protocol is based on the following schema:



There are four main phases:

1. **Nonce exchange phase**: Both parts send to the other a random nonce. As it will be proven in 5, it is necessary to ensure the freshness of the communication and so avoiding replay attacks.

2. **Certificate exchange phase**: Client sends its username and then its public key to the server. Just one of these two information would be enough, but they are both sent to provide a further check on server side. Server then sends back its certificate, signed by a trusted certification authority. It's assumed that the client has the public key of this authority pre installed in a truststore. It's very important that at this point neither client or server have proven their identity. That is, they need to prove themselves owners of the public keys they are declaring.

3. **DH Key Exchange**: The server computes the parameters for Diffie-Hellman Key Exchange (g,p) and send them to the client. Then both the client and the server can compute their private keys, Yc and Ys, and send it to the other party.

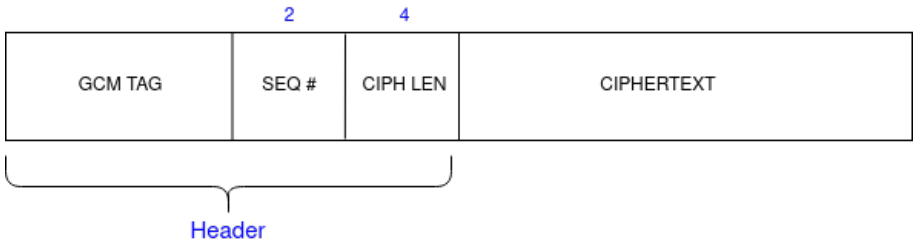
4. **Authentication**: This part is crucial. It is performed together with DH Key Exchange to ensure maximum efficiency. The client must sign the message containing its DH public key: to do so it encrypts a digest with its RSA private key which includes the whole content of M7 (the public key) together with the nonce received by the server (so that freshness can be verified). Vice versa in M8 we can see that the server signs the message by encrypting the digest with the other peer nonce, again to verify the freshness of this interaction. Server certificate is signed by a trusted CA, so there is no possibility that an intruder can falsify the signature.

3.2 Client-Server communication

Before looking at P2P communication between clients it is important to exploit how client and server communicate, both in terms of symmetric encryption and communication protocol.

3.2.1 Symmetric encryption

This paragraph briefly explains the standard format of exchanged packets encrypted using a symmetric algorithm. This standard is the same in both client-server and client-client interactions after the key establishment protocols, so it will be shown just once.

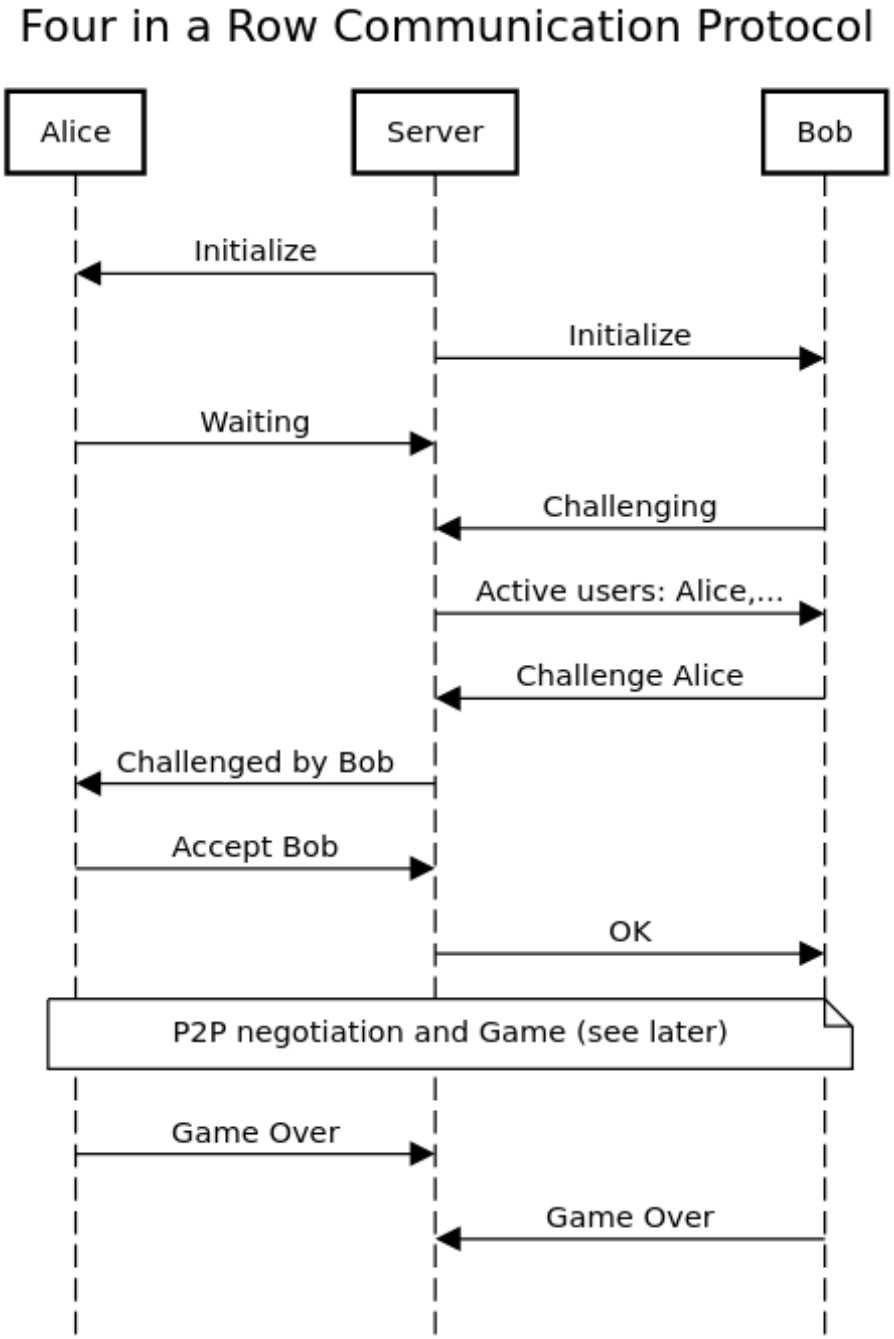


This picture shows the general format of a packet exchanged through the network. The symmetric protocol used is 256 bit AES with authenticated encryption provided by Galois Counter Mode (GCM). Indeed, at the beginning of each packet we have a GCM TAG, whose purpose is to authenticate both the cyphertext and additional data sent in clear. To avoid replay attacks, a sequence number is added to each packet and is sent in plain text. It will be included by the tag, so the receiver can be sure of its authenticity. Of course the value of the sequence number is a counter which is incremented by one at each message exchanged. The last header field contains the ciphertext length (in bytes). It's an important information to simplify the process of receiving whole packets from TCP sockets. The header has a fixed length and all fields but GCM TAG compose the AAD part of GCM envelope. To end the analysis of this envelope, let's talk about the cyphertext. Of course it is encrypted with the symmetric negotiated key, but what it's important to know is how the initialization vector (IV) is derived for each message. In both FRAH Protocol and P2P Handshake an initial IV is negotiated together with the symmetric key. Its value is used to seed a Linear Congruential PRNG (LCG) whose parameters are standard and known to all parties. At each message, a new value for the current IV is computed using the previous value and the LCG, so that each message will have a pseudo random different IV. This standard format is adopted for all messages exchanged with symmetric encryption, and the tag and sequence number will be taken for granted from now on.

3.2.2 Four in a Row Communication Protocol (FRCP)

This paragraph focuses on the semantic of the client-server interaction, providing an overview of the protocol used and a use case of the final application. FRCP is a text based, stateful protocol, meant to provide the maximum readability to messages. For this reason the best way to represent it is through sequential diagrams where each arrow simply includes the packet content. Before looking in detail at the structure of the communication protocol, let's make some points clear:

- Once a user has been authenticated, he/she is asked to choose between waiting for a challenge or challenging someone.
- Depending on what the user has chosen, the interaction is of course different. This is why two diagrams are to be shown.
- When a user chooses to wait for a challenge, the system will inform him/her as soon as a challenge from another user arrives. The user can then accept or decline the incoming challenge.
- At any time the server can send an *initialization* message. In this case the interaction resets to the beginning (right after the handshake protocol). An *initialization* message is also used at the beginning to start the interaction.



The information sent by the server to clients about ip address and public keys are part of P2P handshake protocol, shown in the next section. Note

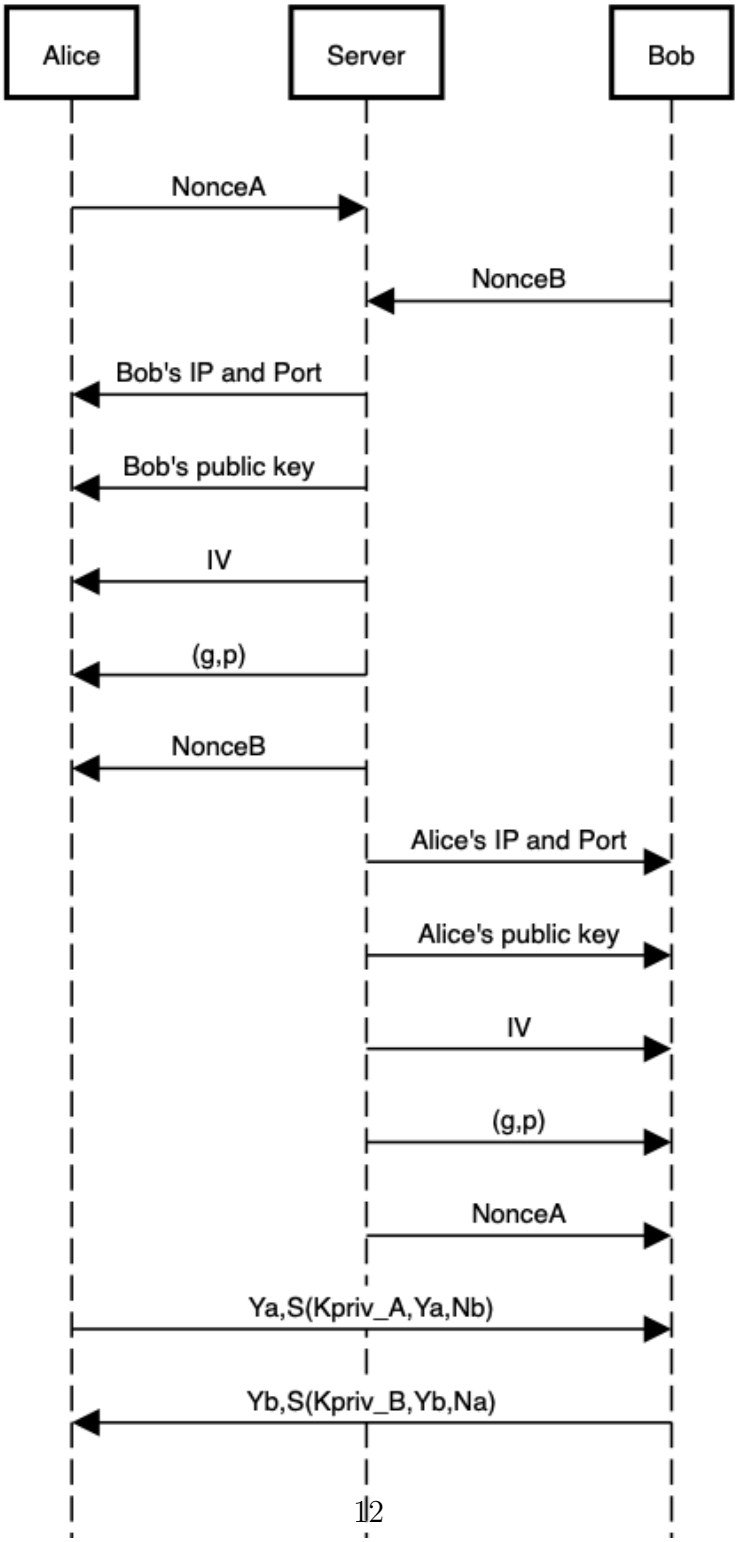
that in case of any format error, sequence number mismatch or signature failure the server simply aborts the connection with the client, preventing further interactions in that session.

3.3 P2P Handshake Protocol

This protocol aims to establish a secure connection between two peers, where secure means both authentication and confidentiality. The server has to provide both clients with the tools to authenticate each other, and it will not know the game key they will share. Note that this P2P symmetric key will be valid just for one game. The protocol is composed on two main phases:

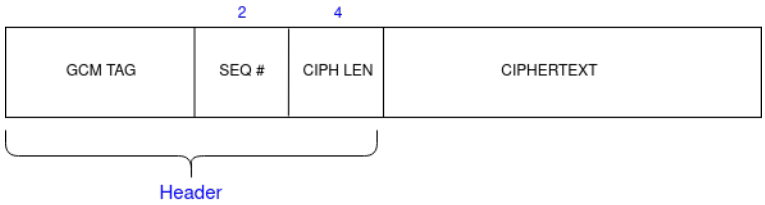
1. **Preparation phase**: The two peers cannot communicate directly from the beginning, because they do not have any information about the other but the username. Since they both consider the server an authority on clients' public keys, it's a server task to forward opponent's public key to each user, so that they can authenticate each other during the real P2P phase. The task to generate a random initialization vector is delegated to the server in this phase, so that Diffie Hellman will be used only to share the symmetric key. IV could also be sent in the clear, but this way possible intruders won't even know it if they attempt an attack. All the messages exchanged between the server and the clients are encrypted with the key established during the FRAH protocol.

2. **P2P phase**: Now the two clients can communicate directly. As we can see, they perform a Diffie Hellman Key Exchange, and then use RSA for the authentication. Indeed, the last two messages provide authentication and freshness. Using the digital signature on the last two messages, we can be safe against replay attack, once that the signatures have been verified.

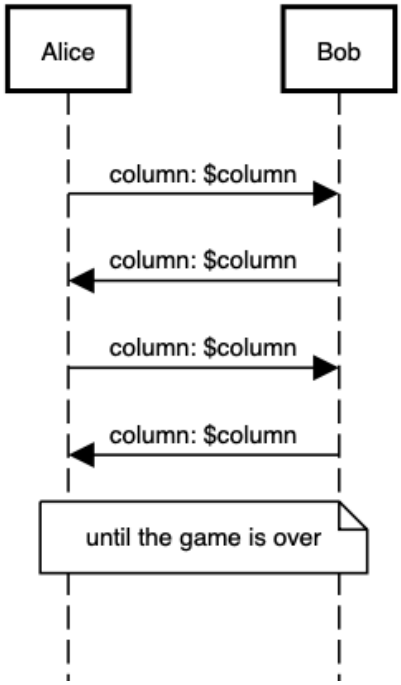

Alice
Server
Bob
NonceA
NonceB
Bob's IP and Port
Bob's public key
IV
(g,p)
NonceB
Alice's IP and Port
Alice's public key
IV
(g,p)
NonceA
Ya,S(Kpriv_A,Ya,Nb)
Yb,S(Kpriv_B,Yb,Na)

4 In-game communication

This section is a conclusion to the exploitment of the protocols used in this application. After establishing a shared key, the two peers can now start their game and will communicate their moves using a simple protocol with symmetric encrypted messages. We'll now show both the packet format and the plaintext.



The structure of messages exchanged between the two clients is exactly the same as the one mentioned before (see 3.2.1). For this reason we will say no more about messages format. The content of messages exchanged during the game can be seen in the following picture:



Once the game is over, both clients will send a Game Over message to the server, as seen above (see 3.2.2).

5 Ban Logic

5.1 FRAH Protocol

A is the client, S the server, C is a CA.
Assumptions:

$$A \mid\equiv C \Rightarrow \overset{Ks}{\longmapsto} S, A \mid\equiv \overset{Kc}{\longmapsto} C$$

$$S \mid\equiv \overset{Ka}{\longmapsto} A, A \mid\equiv \overset{Ka}{\longmapsto} A$$

$$S \mid\equiv \overset{Ks}{\longmapsto} S, C \mid\equiv \overset{Ks}{\longmapsto} S$$

$$A \mid\equiv \#(Na), S \mid\equiv \#(Nb)$$

Objectives:

$$A \mid\equiv A \overset{Kas}{\longleftrightarrow} S, A \mid\equiv \#(A \overset{Kas}{\longleftrightarrow} S), A \mid\equiv S \mid\equiv A \overset{Kas}{\longleftrightarrow} S$$

$$S \mid\equiv A \overset{Kas}{\longleftrightarrow} S, S \mid\equiv \#(A \overset{Kas}{\longleftrightarrow} S), S \mid\equiv A \mid\equiv A \overset{Kas}{\longleftrightarrow} S$$

Idealized Protocol:

M1:

$$A \rightarrow S : Na$$

M2:

$$S \rightarrow A : Nb$$

M3:

$$A \rightarrow S : A$$

M4:

$$A \rightarrow S : \overset{Ka}{\longmapsto} A$$

M5:

$$S \rightarrow A : S, \overset{Ks}{\longmapsto} S, \{S, \overset{Ks}{\longmapsto} S\}_{Kc^{-1}}$$

M6:

$$S \rightarrow A : (g, p)$$

M7:

$$A \rightarrow S : Ya, \{H(Ns, Ya)\}_{Ka^{-1}}$$

M8:

$$S \rightarrow A : Ys, \{H(Nb, Ys)\}_{Ks^{-1}}$$

Proof:
After M5:

$$\frac{A \mid\equiv \overset{Kc}{\longmapsto} C, A \triangleleft \{\overset{Ks}{\longmapsto} S\}_{Kc^{-1}}}{A \mid\equiv C \mid\sim \overset{Ks}{\longmapsto} S}$$

A checks certificate expiration date and CRL, so it's sure that the certificate is fresh:

$$A \mid\equiv \#(\overset{Ks}{\longmapsto} S)$$

Using nonce verification rule:

$$\frac{A \mid\equiv \#(\overset{Ks}{\longmapsto} S), A \mid\equiv C \mid\sim \overset{Ks}{\longmapsto} S}{A \mid\equiv C \mid\equiv \overset{Ks}{\longmapsto} S}$$

So we can say that:

$$\frac{A \mid\equiv C \Rightarrow \overset{Ks}{\longmapsto} S, A \mid\equiv C \mid\equiv \overset{Ks}{\longmapsto} S}{A \mid\equiv \overset{Ks}{\longmapsto} S}$$

After M7:

$$S \triangleleft Ya$$

After M8:

$$A \triangleleft Ys$$

How to authenticate? For DHKE at this point both A,S can compute the shared key:

$$A \mid\equiv A \overset{Kas}{\longleftrightarrow} S, S \mid\equiv A \overset{Kas}{\longleftrightarrow} S$$

and:

$$A \mid\equiv \#(Ya)$$

$$A \mid\equiv \#(Ya, Ys)$$

$$A \mid\equiv \#(A \overset{Kas}{\longleftrightarrow} S)$$

$$S \mid\equiv \#(Ys)$$

$$S \mid\equiv \#(Ya, Ys)$$

$$S \mid\equiv \#(A \overset{Kas}{\longleftrightarrow} S)$$

After M7:

$$\frac{S \lhd \{H(Ya, Nb)\}_{Ka^{-1}}, S \mid\equiv \xmapsto{Ka} A}{S \mid\equiv A \mid\sim H(Ya, Nb)}$$

$$\frac{S \mid\equiv A \mid\sim H(Ya, Nb), S \lhd (Ya, Nb)}{S \mid\equiv A \mid\sim (Ya, Nb)}$$

$$\frac{S \mid\equiv A \mid\sim (Ya, Nb), S \mid\equiv \#(Ya, Nb)}{S \mid\equiv A \mid\equiv (Ya, Nb)}$$

For DHKE we get that

$$S \mid\equiv A \mid\equiv A \overset{Kas}{\longleftrightarrow} S$$

After M7 we can say that

$$A \mid\equiv S \mid\equiv S \overset{Kas}{\longleftrightarrow} S$$

5.2 P2P Handshake Protocol

A,B are the clients, S the server, (g,p) the DH parameters.
Assumptions:

$$A \mid\equiv S \ \Rightarrow \overset{Kb}{\longmapsto} B \ , B \mid\equiv S \ \Rightarrow \overset{Ka}{\longmapsto} A$$

$$A \mid\equiv S \ \Rightarrow \ \#(\overset{Kb}{\longmapsto} B), B \mid\equiv S \ \Rightarrow \ \#(\overset{Ka}{\longmapsto} A)$$

$$A \mid\equiv S \ \Rightarrow \ \#(IVab), B \mid\equiv S \ \Rightarrow \ \#(IVab)$$

$$A \mid\equiv S \ \Rightarrow \ IVab, B \mid\equiv S \ \Rightarrow \ IVab$$

$$A \mid\equiv S \ \Rightarrow \ (g,p), B \mid\equiv S \ \Rightarrow \ (g,p)$$

$$Kab = g^{ab} mod p$$

$$B \mid\equiv \ B \overset{Kbs}{\longleftrightarrow} S, B \mid\equiv \ \#(B \overset{Kbs}{\longleftrightarrow} S)$$

$$A \mid\equiv \ A \overset{Kas}{\longleftrightarrow} S, A \mid\equiv \ \#(A \overset{Kas}{\longleftrightarrow} S)$$

$$A \mid\equiv \ \#(Ya) \, B \mid\equiv \ \#(Yb) \ , A \mid\equiv \ \#(Nas) \ , B \mid\equiv \ \#(Nbs)$$

Objectives:

$$A \mid\equiv \ A \overset{Kab}{\longleftrightarrow} B, A \mid\equiv \ \#(A \overset{Kab}{\longleftrightarrow} B) \ , A \mid\equiv \ B \mid\equiv \ A \overset{Kab}{\longleftrightarrow} B$$

$$B \mid\equiv \ A \overset{Kab}{\longleftrightarrow} B, B \mid\equiv \ \#(A \overset{Kab}{\longleftrightarrow} B) \ , B \mid\equiv \ A \mid\equiv \ A \overset{Kab}{\longleftrightarrow} B$$

Idealized protocol:
Pre-phase:

M1:

$$A \rightarrow S : \{Nas\}_{Kas}$$

$$B \rightarrow S : \{Nbs\}_{Kbs}$$

M2:

$$S \rightarrow A : \{\overset{Kb}{\longmapsto} B \ , \#(\ \overset{Kb}{\longmapsto} B \ ) \ \}_{Kas}$$

$$S \rightarrow B : \{\overset{Ka}{\longmapsto} A \ , \#(\ \overset{Ka}{\longmapsto} A \ ) \ \}_{Kbs}$$

M3:

$$S \rightarrow A : \{IVab, \#(IVab) \ \}_{Kas}$$

$$S \rightarrow B : \{IVab, \#(IVab)\ \}_{Kbs}$$

M4:

$$S \rightarrow A : \{g, p\}_{Kas}$$
$$S \rightarrow B : \{g, p\}_{Kbs}$$

M5:

$$S \rightarrow A : \{Nbs\}_{Kas}$$
$$S \rightarrow B : \{Nas\}_{Kbs}$$

P2P-phase:

M6:

$$A \rightarrow B : Ya = g^a modp, \{H(Ya, Nbs)\}_{Ka^{-1}}$$

M7:

$$B \rightarrow A : Yb = g^b modp, \{H(Yb, Nas)\}_{Kb^{-1}}$$

Proof:
In the pre-phase we know for assumption that both A,B believe in their symmetric key with the server. So, we skip for brevity the following postulate:

$$\frac{P \mid\equiv\ Q \overset{K}{\leftrightarrow} P \lhd \{X\}_K}{P \lhd X}$$

in all pre-phase. After M2: After receiving Nas (Nbs), S said that Kb is B's public key (Ka is A's public key).

$$\frac{A \mid\equiv S \mid\equiv \overset{Kb}{\longmapsto} B, A \mid\equiv S \Rightarrow \overset{Kb}{\longmapsto} B}{A \mid\equiv \overset{Kb}{\longmapsto} B}$$

$$\frac{A \mid\equiv S \mid\equiv \#(\overset{Kb}{\longmapsto} B), A \mid\equiv S \Rightarrow \#(\overset{Kb}{\longmapsto} B)}{A \mid\equiv \#(\overset{Kb}{\longmapsto} B)}$$

For the same jurisdiction rule, we have:

$$B \mid\equiv \overset{Ka}{\longmapsto} A, B \mid\equiv \#(\overset{Ka}{\longmapsto} A)$$

After M3: Again for nonce verification both:

$$A \mid\equiv S \mid\equiv IVab, A \mid\equiv S \mid\equiv \#(IVab)$$

$$B \mid\equiv S \mid\equiv IVab, B \mid\equiv S \mid\equiv \#(IVab)$$

And again, for jurisdiction rule:

$$A \mid\equiv IVab, A \mid\equiv \#(IVab)$$

$$B \mid\equiv IVab, B \mid\equiv \#(IVab)$$

After M4: For jurisdiction rule, we have:

$$A \mid\equiv g, b$$

$$B \mid\equiv g, b$$

After M6:

$$A \triangleleft Yb$$

(A doesn't know who sent Yb)

After M7:

$$B \triangleleft Ya$$

(B doesn't know who sent Ya)

How to authenticate? For DHKE at this point both A,B can compute the shared key:

$$A \mid\equiv A \overset{Kab}{\longleftrightarrow} B, B \mid\equiv A \overset{Kab}{\longleftrightarrow} B$$

and:

$$A \mid\equiv \#(Ya), A \mid\equiv \#(IVab)$$

$$A \mid\equiv \#(Ya, Yb, IVab)$$

$$A \mid\equiv \#(A \overset{Kab}{\longleftrightarrow} B)$$

$$B \mid\equiv \#(Yb), B \mid\equiv \#(IVab)$$

$$B \mid\equiv \#(Ya, Yb, IVab)$$

$$B \mid\equiv \#(A \overset{Kab}{\longleftrightarrow} B)$$

After M6:

$$\frac{B \triangleleft \{H(Ya, Nbs)\}_{Ka^{-1}}, B \mid\equiv \overset{Ka}{\longmapsto} A}{B \mid\equiv A \mid\sim H(Ya, Nbs)}$$

$$\frac{B \mid\equiv A \mid\sim H(Ya, Nbs), B \lhd (Ya, Nbs)}{B \mid\equiv A \mid\sim (Ya, Nbs)}$$

$$\frac{B \mid\equiv A \mid\sim (Ya, Nbs), B \mid\equiv \#(Ya, Nbs)}{B \mid\equiv A \mid\equiv (Ya, Nbs)}$$

For DHKE we get that

$$B \mid\equiv A \mid\equiv A \overset{Kab}{\longleftrightarrow} B$$

After M7 we can say that

$$A \mid\equiv B \mid\equiv A \overset{Kab}{\longleftrightarrow} B$$

with the only difference that freshness is given by:

$$\frac{A \mid\equiv B \mid\sim (Yb, Nas), A \mid\equiv \#(Yb, Nas)}{A \mid\equiv B \mid\equiv (Yb, Nas)}$$